

React Native

Learning Field Guide

The Missed Stuff — Gaps From Parts 1-3

Windows · Expo SDK 54 · Android

Concepts used in every single file that were never properly explained.
StyleSheet, flex layout, export patterns, TextInput props, Supabase query builder, the profiles table, and EarGuard-specific Android prep.

#01	StyleSheet.create()	Why not just write styles inline?
#02	flex: 1 and Flexbox	The layout system behind every screen
#03	SafeAreaView vs View	What safe areas are and why they matter
#04	export default vs export const	The difference and when to use each
#05	TextInput props deep dive	autoCapitalize, autoComplete, secureTextEntry
#06	Supabase query builder	.from().select().eq().single() explained
#07	Profiles table + auth users	Why two tables exist for one user
#08	AsyncStorage + JWT tokens	What actually gets stored when you log in
#09	Android foreground services	What EarGuard needs and why it is different
#10	Native modules vs JS packages	The difference that drives your build process

What's In Part 4

These are concepts you used in Parts 1-3 that were never properly explained. StyleSheet, flex layout, and the Supabase query builder appear in literally every file you wrote. Not understanding them is like building a house without knowing what a wall is.

#01 StyleSheet.create()

Why not write styles inline?

#02 flex: 1 and Flexbox

The layout system behind every screen

#03 SafeAreaView vs View

What safe areas are and why they matter

#04 export default vs export const

When to use which

#05 TextInput props deep dive

Every prop you used, explained

#06 Supabase query builder chain

`.from().select().eq().single()`

#07 Profiles table vs auth users

Why two tables exist for one user

#08 AsyncStorage + JWT tokens

What actually gets stored on login

#09 Android foreground services

What EarGuard needs and why

#10 Native modules vs JS packages

The difference that drives your builds

#01

StyleSheet.create()

Used in every single file

What is it?

StyleSheet is a React Native API that creates an optimised style object. Every file you wrote ends with a `StyleSheet.create()` call. It looks like CSS but it is just a JavaScript object with a thin wrapper around it. React Native maps these style properties to native Android layout attributes at runtime.

```
// Why StyleSheet.create() instead of plain objects
// OPTION 1: inline styles (works but bad)
<View style={{ flex: 1, backgroundColor: "#fff" }}>
// Problem: a NEW object is created on EVERY render.
// Every re-render = new object = React thinks style changed
// = unnecessary layout recalculation.
// For simple one-off styles: fine.
// For shared or repeated styles: performance hit.
// OPTION 2: plain object outside component (better)
const styles = {
  container: { flex: 1, backgroundColor: "#fff" }
};
// Created once. Never recreated. No performance hit.
// But no validation – typos in property names are silent.
// OPTION 3: StyleSheet.create() (best)
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#fff" },
  title: { fontSize: 32, fontWeight: "bold" },
});
// Created once. Validated at creation time.
// StyleSheet.create() checks that property names are valid RN styles.
// Invalid property? Warning in dev mode. Caught early.
```

```
// StyleSheet from your login.tsx – annotated
const styles = StyleSheet.create({
  container: {
    flex: 1, // takes all available space
  },
  content: {
    flex: 1,
    justifyContent: "center", // centre children vertically
    padding: 24, // 24px on all sides
  },
  input: {
    backgroundColor: "#f5f5f5",
    borderRadius: 12, // rounded corners (12px radius)
    padding: 16, // internal spacing
    fontSize: 16,
    marginBottom: 16, // space below each input
    borderWidth: 1, // 1px border
    borderColor: "#e0e0e0",
  },
  button: {
    backgroundColor: "#000",
    borderRadius: 12,
    padding: 16,
    alignItems: "center", // centre children horizontally
  },
});
// Usage: style={styles.container}
// The styles object is created ONCE when the module loads.
// Not recreated on every render.
```

React Native styles vs CSS — key differences

No cascading. Styles do not inherit from parent to child (except Text inside Text). No class selectors. No IDs. No pseudo-selectors (:hover, :focus). All values are numbers (not strings with 'px'): padding: 16 not padding: '16px'. Exception: percentage strings for some layout props: width: '100%'. camelCase instead of kebab-case: backgroundColor not background-color.

#02

flex: 1 and Flexbox

The layout system behind every screen

What is Flexbox?

React Native uses Flexbox as its ONLY layout system. There is no CSS grid, no floats, no absolute positioning by default. Everything is Flexbox. Unlike web CSS where Flexbox is opt-in, in React Native every View is already a flex container by default. The default flex direction in React Native is COLUMN (top to bottom), not row like in web CSS — this catches everyone.

```
// flex: 1 – what it actually means
// flex: 1 means: take up ALL remaining space in the parent.
// Example from your code:
container: { flex: 1 }
content: { flex: 1, justifyContent: "center", padding: 24 }
// The screen has a fixed height (e.g. 800px).
// container with flex: 1 = fill the entire 800px.
// content inside it with flex: 1 = fill all of containers space.
// With multiple flex children, the space is DIVIDED:
header: { flex: 1 } // gets 1/3 of parent height
body: { flex: 2 } // gets 2/3 of parent height
footer: { flex: 1 } // gets 1/3 of parent height
// Without flex: 1, a View only takes as much space as its content.
// justifyContent: "center" on a View with no height = does nothing.
// This is why screens look broken when flex: 1 is missing.
```

```
// justifyContent vs alignItems – the axis confusion
// Default flexDirection is "column" (vertical stacking).
// This flips which axis justifyContent and alignItems control.
// justifyContent = controls the MAIN axis
// In column mode: main axis = VERTICAL (up/down)
justifyContent: "center" // centre children vertically
justifyContent: "flex-start" // push to top
justifyContent: "flex-end" // push to bottom
justifyContent: "space-between" // spread with space between
// alignItems = controls the CROSS axis
// In column mode: cross axis = HORIZONTAL (left/right)
alignItems: "center" // centre children horizontally
alignItems: "flex-start" // push to left
alignItems: "stretch" // stretch to fill width (default)
// From your onboarding.tsx:
form: {
  width: "100%",
  alignItems: "center", // centres the image and inputs horizontally
}
```

The most common flex mistake — forgetting flex: 1 on the container

If your screen content appears squished at the top and `justifyContent: 'center'` seems to do nothing, the container is missing `flex: 1`. Without `flex: 1`, the View has zero height (just enough for its content). You cannot centre inside a zero-height container. Fix: add `flex: 1` to the container style.

#03

SafeAreaView vs View

What safe areas are and why they matter

What is a safe area?

Modern phones have areas of the screen that are physically blocked or reserved: the status bar at the top (where your time and battery live), the navigation bar at the bottom (home button / gesture bar), and on iPhones the notch or Dynamic Island. If you render content behind these areas, it gets hidden or overlaps system UI. `SafeAreaView` automatically adds padding to keep your content in the visible safe zone.

```
// SafeAreaView vs View – the practical difference
import { View } from "react-native";
// Plain View: renders content from edge to edge of the screen.
// Your title text could hide behind the Android status bar.
// On devices with gesture navigation: bottom content cut off.
import { SafeAreaView } from "react-native-safe-area-context";
// SafeAreaView: automatically adds insets (padding) to avoid:
// - Status bar at top
// - Navigation bar at bottom
// - Notch / Dynamic Island on iOS
// Your pattern (used in every auth screen):
return (
  <SafeAreaView style={styles.container}>
    <View style={styles.content}>
      // content here is safe from system UI overlap
    </View>
  </SafeAreaView>
);
// Why react-native-safe-area-context and not the built-in one?
// The built-in SafeAreaView (from react-native) is iOS-only.
// react-native-safe-area-context works on BOTH Android and iOS.
// Always use the library version.
```

When to use SafeAreaView vs View

`SafeAreaView`: use on ROOT screen containers — the outermost wrapper of each screen. `View`: use for everything else inside the screen. You do not need to wrap every `View` in `SafeAreaView` — just the top-level screen container. Using `SafeAreaView` for nested Views adds unnecessary extra padding.

#04

export default vs export const

The difference and when to use each

What is the difference?

#05

TextInput Props Deep Dive

Every prop you used, explained

What are these props?

TextInput is the React Native equivalent of an HTML input field. It has many props that control keyboard type, autocomplete behaviour, capitalisation, and security. You used most of these in login, signup, and onboarding without a full explanation of what each one does.

```
// Every TextInput prop from your codebase – annotated
<TextInput
  placeholder="Email..."
  // Ghost text shown when the field is empty.
  // Disappears when user starts typing.
  placeholderTextColor={"#999"}
  // Colour of the placeholder text.
  // Default is system grey which is often invisible on some themes.
  keyboardType="email-address"
  // Shows @ and . on the keyboard for easier email entry.
  // Other values: "numeric", "phone-pad", "url", "default"
  autoComplete="email"
  // Tells Android to offer email autocomplete suggestions.
  // Other values: "password", "username", "name", "tel"
  autoCapitalize="none"
  // Prevents auto-capitalising the first letter.
  // Critical for email and username fields.
  // Other values: "sentences", "words", "characters"
  // Your onboarding.tsx uses "words" for the Full Name field.
  secureTextEntry
  // Hides characters as dots (password masking).
  // Also disables autocomplete for security.
  // Shorthand for secureTextEntry={true}.
  value={email}
  // Controlled input: the displayed value is driven by state.
  // Without this: TextInput is uncontrolled (you cannot read it).
  onChangeText={setEmail}
  // Called every time the text changes.
  // Receives the new string value.
  // setEmail is your useState setter – updates state on every keypress.
/>
```

Controlled vs uncontrolled inputs

A controlled input has `value={state}` and `onChangeText={setter}`. The input displays whatever is in state, and updates state on every keypress. You are always in control of the value. An uncontrolled input has no value prop — React Native manages the value internally and you cannot easily read it. Always use controlled inputs for forms. Your `login.tsx` originally had uncontrolled inputs which is why the form captured nothing.

#06

Supabase Query Builder Chain

`.from().select().eq().single()`

What is a query builder?

Instead of writing raw SQL, Supabase gives you a JavaScript API that builds the SQL query for you through chained method calls. Each method adds a clause to the query. The chain is only executed when you await it — calling `.from()` or `.select()` alone does nothing until awaited.

```
// The query from onboarding.tsx – broken down
const { data: existingUser } = await supabase
  .from("profiles")
  .select("id")
  .eq("username", username)
  .single();
// Let us break each method down:
// .from("profiles")
// Which table to query. Equivalent to: FROM profiles in SQL.
// .select("id")
// Which columns to return. "id" = only fetch the id column.
// .select("*") = fetch all columns.
// .select("id, username, name") = fetch specific columns.
// Fetching only what you need = faster query, less data transfer.
// .eq("username", username)
// Add a WHERE condition. "column", value.
// Equivalent to: WHERE username = 'gautam' in SQL.
// Other filters: .neq() .gt() .lt() .like() .in()
// .single()
// Expect exactly 0 or 1 result (not an array).
// Returns the object directly, not wrapped in an array.
// If 0 results: data = null (no error thrown).
// If 2+ results: throws an error (unexpected duplicate).
// { data: existingUser }
// Rename data to existingUser for clarity.
// This is JavaScript destructuring rename syntax:
// const { data: newName } = someObject
```

```
// More query patterns you will use in EarGuard
// INSERT a new row
const { data, error } = await supabase
  .from("profiles")
  .insert({
    id: userId,
    username: "gautam",
    name: "Gautam",
  });
// UPDATE a row
const { data, error } = await supabase
  .from("profiles")
  .update({ onboarding_completed: true })
  .eq("id", userId);
// SELECT with multiple conditions
const { data, error } = await supabase
  .from("sessions")
  .select("*")
  .eq("user_id", userId)
  .gt("duration_seconds", 300) // greater than 5 minutes
  .order("created_at", { ascending: false }); // newest first
// Real-time subscription (you will need this for EarGuard)
const channel = supabase
  .channel("alert-channel")
  .on("postgres_changes", { event: "INSERT", schema: "public" },
    (payload) => { console.log(payload.new); }
  )
  .subscribe();
```

#07

Profiles Table vs Auth Users

Why two tables exist for one user

Why does this pattern exist?

Supabase auth creates users in a special internal table called `auth.users`. This table is managed by Supabase and stores auth-specific data: email, hashed password, email confirmation status, login metadata. You cannot add custom columns to `auth.users`. So when you need to store app-specific user data (username, display name, profile picture), you create a separate `public.profiles` table and link it by user ID.

```
// The two-table architecture
// TABLE 1: auth.users (managed by Supabase, you cannot modify)
// {
// id: "uuid-here", // the user UUID
// email: "g@test.com",
// encrypted_password: "bcrypt-hash",
// email_confirmed_at: "2026-05-01T...",
// created_at: "2026-05-01T...",
// }
// TABLE 2: public.profiles (YOUR table – add whatever you need)
// {
// id: "uuid-here", // SAME as auth.users.id
// name: "Gautam",
// username: "gautam_dev",
// profile_picture_url: "https://...",
// onboarding_completed: true,
// }
// The id column in profiles is a FOREIGN KEY to auth.users.id.
// One user = one row in auth.users + one row in profiles.
// When you create a user via supabase.auth.signUp(),
// Supabase creates the auth.users row automatically.
// YOU create the profiles row in handleComplete() in onboarding.tsx.
```

```
// Reading current user data
// Get the currently logged in user from Supabase auth:
const { data: { user } } = await supabase.auth.getUser();
// user.id = the UUID you use to link to profiles table
// user.email = their email address
// Get their profile data:
const { data: profile } = await supabase
  .from("profiles")
  .select("*")
  .eq("id", user.id)
  .single();
// profile.username, profile.name, profile.profile_picture_url
// This is what setUser() in AuthContext should do:
// After signIn(), call getUser(), then fetch their profile,
// then map both into your User interface and call setUser().
```

#08

AsyncStorage + JWT Tokens

What actually gets stored when you log in

What is a JWT token?

When you log in via Supabase, the server generates a JSON Web Token (JWT). This is a long encoded string that proves who you are to the server on every subsequent request. Think of it as a temporary digital ID card. It contains your user ID and an expiry time, and it is cryptographically signed so the server can verify it has not been tampered with.

```
// What Supabase stores in AsyncStorage
// After a successful signIn(), Supabase stores this in AsyncStorage:
// Key: "sb-[project-ref]-auth-token"
// Value: JSON string containing:
// {
//   access_token: "eyJhbGc...", // JWT (expires in ~1 hour)
//   refresh_token: "abc123...", // used to get new access_token
//   expires_at: 1234567890, // Unix timestamp of expiry
//   user: {
//     id: "uuid",
//     email: "g@test.com",
//   }
// }
// autoRefreshToken: true in client.ts means:
// Before access_token expires, Supabase automatically uses
// the refresh_token to get a new access_token.
// Users stay logged in without needing to re-enter credentials.
// persistSession: true means:
// This JSON is written to AsyncStorage (permanent storage).
// When app restarts, Supabase reads it back and restores the session.
// This is why "remember me" works without you writing any extra code.
```

Why AsyncStorage and not regular variables

Regular JavaScript variables live in memory — they are gone the instant the app closes. AsyncStorage persists data to the device's internal storage, like a tiny database. It is asynchronous (hence the name) — reads and writes return Promises. Supabase handles all the AsyncStorage calls for you through the storage: AsyncStorage config in client.ts. You never need to read or write to it directly.

#09

Android Foreground Services

What EarGuard needs and why it is different

What is a foreground service?

Android aggressively kills background processes to save battery. If EarGuard is monitoring audio while the user is on another app, Android will eventually kill it. A foreground service is Android's way of saying: this app is actively doing something important — do not kill it. It requires showing a persistent notification to the user so they always know something is running in the background.

```
// Why EarGuard specifically needs a foreground service
// EarGuard needs to:
// 1. Monitor audio input continuously
// 2. Track cumulative listening duration
// 3. Send an alert when limits are exceeded
// 4. All while the user might be on another app
// Without a foreground service:
// - Android kills EarGuard after ~1-2 minutes in background
// - Monitoring stops
// - No alerts are sent
// - The whole app is useless
// With a foreground service:
// - Android keeps EarGuard alive in background
// - User sees a persistent notification: "EarGuard monitoring..."
// - Monitoring continues indefinitely
// - Alerts fire correctly
// In React Native you will use:
// react-native-foreground-service (or a similar library)
// This is a NATIVE MODULE – requires a full rebuild after install.
// Remember: npx expo run:android after installing.
```

Data warning: foreground service setup requires a full rebuild

react-native-foreground-service is a native module (Java code on Android). Installing it with `npx expo install` and then running `npx expo run:android` will trigger a full Gradle rebuild. Expect 200-400MB of downloads plus build time. WiFi only.

```
// How foreground service maps to patterns you already know
// You already know useEffect with cleanup:
useEffect(() => {
  startMonitoring(); // start the foreground service
  return () => {
    stopMonitoring(); // cleanup: stop when component unmounts
  };
}, []);
// You already know Context:
// MonitoringContext will hold: isMonitoring, decibelLevel, totalMinutes
// Exactly like AuthContext holds: user, signIn, signUp
// You already know useRef:
// The audio recorder instance will be held in a useRef
// Exactly like bottomSheetRef held the BottomSheet instance
// The patterns are identical. The native layer is new.
// Everything above the native layer is React Native you already know.
```


Quick Reference — Native vs JS

Package	Type	After install
@supabase/supabase-js	Pure JS	npx expo start
expo-router	Pure JS	npx expo start
expo-image-picker	Native	npx expo run:android
react-native-gesture-handler	Native	npx expo run:android
@gorhom/bottom-sheet	Native	npx expo run:android
@react-native-async-storage	Native	npx expo run:android
react-native-reanimated	Native	npx expo run:android
@expo/vector-icons	Pure JS	npx expo start

The Complete Series — What You Now Know

Part 1 — Concept Glossary

useState, useEffect, Context API, TypeScript, JSX, async/await, Expo Router

Part 2 — Code Walkthrough

All 10 files line by line — client.ts, AuthContext, layouts, screens, BottomSheet

Part 3 — Bugs + Prevention

9 bugs with root cause analysis, 10 prevention laws, command cheatsheet

Part 4 — The Missed Stuff

StyleSheet, Flexbox, SafeAreaView, exports, TextInput, Supabase queries, EarGuard prep

You went from zero React Native knowledge to understanding authentication flows, Context architecture, file-based routing, Supabase integration, gesture handling, image picking, and the native vs JS build distinction. That is not beginner material. That is intermediate React Native. EarGuard is within reach.

Immediate next steps before starting EarGuard

1. Fix the 8 active bugs in your current files (list in Part 3). 2. Implement setUser() in AuthContext so the auth flow actually works end to end. 3. Test the full signup to onboarding to home tab flow on your physical device. 4. Complete the BeReal clone tutorial to build camera and feed features. 5. THEN start EarGuard — you will have the foundation you need.

Spanish word of the day: aprender

aprender (ah-pren-DEHR) = to learn. Example: Estoy aprendiendo React Native. (I am learning React Native.) Four PDFs later and you are doing exactly that. Buen trabajo.